

Principal Component Analysis

Theory, Implementation, and Clinical Application



Mediacrest Training College

Advanced Data Analysis and Fundamentals of AI

Week 8: Unsupervised Learning - Dimensionality Reduction

Author:

Victor Wandera Lumumba, MSc.
Statistician | Data Analyst | ML Expert

October 5, 2026

Contents

1	Introduction to Dimensionality Reduction	5
1.1	The Curse of Dimensionality	5
1.2	What is Dimensionality Reduction?	5
1.3	Objectives of PCA	6
2	Theoretical Foundations of PCA	7
2.1	Mathematical Intuition	7
2.2	Variance and Covariance	7
2.3	Eigenvalues and Eigenvectors	7
2.4	Steps in PCA Algorithm	8
3	Data Preprocessing and Exploration	9
3.1	Dataset Overview: CVD Risk	9
3.1.1	Features	9
3.1.2	Target Variable	9
3.2	Loading Libraries	9
3.3	Exploratory Data Analysis (EDA)	10
3.3.1	Interpretation of Correlation Heatmap	10
3.4	Train-Test Split Strategy	10
3.5	Feature Standardization	11
4	PCA Implementation and Component Selection	13
4.1	Fitting PCA on All Components	13
4.2	Explained Variance Ratio	13
4.2.1	Results	13
4.3	Determining Optimal Components	13
4.4	Visualizing Variance: The Scree Plot	14
4.4.1	Interpretation	14
4.5	2D PCA Projection	15
5	Interpretation of Principal Components	17
5.1	Understanding Loadings	17
5.2	Calculating Loadings	17
5.3	Loadings Table	17
5.4	Interpreting PC1: Cardiometabolic Risk	17
5.5	Interpreting PC2: Inflammation	18
5.6	Interpreting PC3: Age	18
5.7	Visualizing Loadings	18
6	Model Evaluation: Raw vs. PCA Features	19

6.1	Baseline Model: Logistic Regression (Raw)	19
6.1.1	Baseline Results	19
6.2	PCA Model: Logistic Regression (PCA)	19
6.2.1	PCA Model Results	20
6.3	Comparative Analysis	20
6.3.1	Key Findings	20
6.4	Classification Report Details	21
7	Conclusion and Best Practices	23
7.1	Summary	23
7.2	Best Practices for PCA	23
7.3	Limitations	23
7.4	Future Directions	24

Chapter 1

Introduction to Dimensionality Reduction

1.1 The Curse of Dimensionality

In modern data science, datasets often contain hundreds or thousands of features. While more data can sometimes lead to better models, high-dimensional data introduces significant challenges known as the *Curse of Dimensionality*.

As the number of dimensions increases, the volume of the space increases so fast that the available data become sparse. This sparsity is problematic for any method that requires statistical significance. In order to obtain a statistically sound and reliable result, the amount of data needed to support the result often grows exponentially with the dimensionality.

Key issues include:

- **Overfitting:** Models may learn noise rather than signal.
- **Computational Cost:** Algorithms become slower and memory-intensive.
- **Multicollinearity:** Features are often correlated, violating assumptions of linear models.
- **Visualization Difficulty:** Humans cannot visualize data beyond three dimensions.

1.2 What is Dimensionality Reduction?

Dimensionality reduction is the transformation of data from a high-dimensional space into a low-dimensional space so that the low-dimensional representation retains some meaningful properties of the original data, ideally close to its intrinsic dimension.

There are two main approaches:

1. **Feature Selection:** Selecting a subset of relevant features from the original set (e.g., Filter methods, Wrapper methods).
2. **Feature Extraction:** Creating new composite features that are combinations of the original features (e.g., PCA, LDA, t-SNE).

This book focuses on **Principal Component Analysis (PCA)**, the most widely used feature extraction technique.

1.3 Objectives of PCA

Principal Component Analysis (PCA) is a multivariate statistical technique used to reduce the dimensionality of a dataset while retaining as much information as possible. Its primary objectives are:

- Reduce the number of variables in a dataset.
- Remove multicollinearity among predictors by creating orthogonal components.
- Retain maximum variation present in the original data.
- Simplify data interpretation and visualization.
- Improve computational efficiency in predictive modeling.

Chapter 2

Theoretical Foundations of PCA

2.1 Mathematical Intuition

PCA creates new variables known as **Principal Components (PCs)**. These components are linear combinations of the original variables and are arranged according to the amount of variance they explain.

Let X be a data matrix with n observations and p features. We seek to find a new coordinate system such that:

1. The first axis (PC1) captures the largest possible variance in the data.
2. The second axis (PC2) captures the second-largest variance and is orthogonal (perpendicular) to PC1.
3. Subsequent components explain progressively smaller amounts of variance.

2.2 Variance and Covariance

Variance measures how spread out the data is. Covariance measures how two variables change together. PCA relies on the **Covariance Matrix** Σ :

$$\Sigma = \frac{1}{n-1} X^T X \quad (2.1)$$

Where X is the centered data matrix (mean subtracted). The diagonal elements of Σ represent the variances of individual features, and the off-diagonal elements represent the covariances between pairs of features.

2.3 Eigenvalues and Eigenvectors

The mathematical foundation of PCA is based on the decomposition of the covariance matrix through eigenvalues and eigenvectors.

For a square matrix A , an eigenvector v and eigenvalue λ satisfy:

$$Av = \lambda v \quad (2.2)$$

In the context of PCA:

- **Eigenvectors:** Represent the directions of the new feature space (the Principal Components). They define the orientation of the axes.
- **Eigenvalues:** Represent the magnitude of variance captured by each eigenvector. Larger eigenvalues indicate more important components.

2.4 Steps in PCA Algorithm

The standard procedure for performing PCA involves the following steps:

1. **Standardize the Data:** Scale features to have mean 0 and variance 1. This is critical because PCA is sensitive to the scale of variables.
2. **Compute the Covariance/Correlation Matrix:** Identify relationships between variables.
3. **Calculate Eigenvalues and Eigenvectors:** Decompose the matrix to find directions of maximum variance.
4. **Rank Principal Components:** Sort eigenvectors by their corresponding eigenvalues in descending order.
5. **Select Important Components:** Choose the top k components based on explained variance (e.g., 90% threshold).
6. **Transform Data:** Project the original data onto the new component space using the selected eigenvectors.

Chapter 3

Data Preprocessing and Exploration

3.1 Dataset Overview: CVD Risk

We utilize the Cardiovascular Disease (CVD) Risk dataset, which contains clinical markers for patients. The goal is to predict CVD risk while reducing the complexity of the input features.

3.1.1 Features

- Systolic_BP: Systolic Blood Pressure
- Diastolic_BP: Diastolic Blood Pressure
- Total_Cholesterol: Total Cholesterol levels
- HDL_Cholesterol: High-Density Lipoprotein
- Fasting_Glucose: Blood glucose levels
- CRP_Inflammation: C-Reactive Protein (inflammation marker)
- Age: Patient age
- LDL_Cholesterol: Low-Density Lipoprotein

3.1.2 Target Variable

- CVD_Risk: Binary variable (0 = Low Risk, 1 = High Risk)

3.2 Loading Libraries

To perform PCA and subsequent modeling, we require the following Python libraries:

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
```

```
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.decomposition import PCA
7 from sklearn.model_selection import train_test_split,
  cross_val_score
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.metrics import accuracy_score, roc_auc_score,
  classification_report
10 import warnings
11 warnings.filterwarnings('ignore')
12 plt.style.use('seaborn-v0_8-whitegrid')
```

Listing 3.1: Loading Required Libraries

3.3 Exploratory Data Analysis (EDA)

Before applying PCA, it is crucial to understand the relationships between variables. High correlations suggest redundancy, making PCA an ideal candidate for dimensionality reduction.

```
1 features = df.columns[:-1]
2 corr_matrix = df[features].corr()
3 plt.figure(figsize=(10, 7))
4 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1,
  vmax=1,
5             fmt='.2f', square=True, linewidths=0.5)
6 plt.title('Correlation Matrix: High Multicollinearity Expected')
7 plt.tight_layout()
8 plt.show()
```

Listing 3.2: Correlation Analysis

3.3.1 Interpretation of Correlation Heatmap

The heatmap reveals strong positive correlations ($|r| > 0.7$) among Blood Pressure markers, Cholesterol levels, and Glucose. This multicollinearity indicates that these variables share common underlying information. PCA will help disentangle these shared variations into independent components.

3.4 Train-Test Split Strategy

To prevent data leakage, we must split the data *before* scaling.

```
1 X = df[features].values
2 y = df['CVD_Risk'].values
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y,
6     test_size=0.25,
7     random_state=42,
```

```
8 stratify=y)
```

Listing 3.3: Train-Test Split

Key Parameters:

- `test_size=0.25`: 25% of data is held out for final evaluation.
- `stratify=y`: Ensures the proportion of High/Low risk patients is preserved in both sets.
- `random_state=42`: Ensures reproducibility.

3.5 Feature Standardization

PCA is variance-sensitive. If one feature has a range of 0–100 and another 0–1, the first will dominate the principal components. Therefore, standardization is mandatory.

```
1 scaler = StandardScaler()  
2 # Fit ONLY on training data to prevent leakage  
3 X_train_scaled = scaler.fit_transform(X_train)  
4 # Transform test data using training parameters  
5 X_test_scaled = scaler.transform(X_test)  
6 print("    Data split & standardized (mean=0, variance=1).")
```

Listing 3.4: Standardization

Chapter 4

PCA Implementation and Component Selection

4.1 Fitting PCA on All Components

Initially, we fit PCA on all components to evaluate how much variance each explains. This helps us decide how many components to retain.

```
1 pca_full = PCA(random_state=42)
2 X_train_pca_full = pca_full.fit_transform(X_train_scaled)
```

Listing 4.1: Initial PCA Fit

4.2 Explained Variance Ratio

The `explained_variance_ratio_` attribute tells us the proportion of the dataset's total variance that is attributable to each individual component.

```
1 explained_var = pca_full.explained_variance_ratio_
2 cumulative_var = np.cumsum(explained_var)
3
4 print(f"Explained Variance per Component: {np.round(
5     explained_var, 3)}")
6 print(f"Cumulative Variance: {np.round(cumulative_var, 3)}")
```

Listing 4.2: Calculating Explained Variance

4.2.1 Results

4.3 Determining Optimal Components

We aim to retain enough components to explain at least 90% of the total variance.

```
1 threshold = 0.90
2 n_components_opt = np.argmax(cumulative_var >= threshold) + 1
3 print(f"Optimal Components for {threshold*100}% Variance
4     : {n_components_opt}")
```

Component	Individual Variance	Cumulative Variance
PC1	0.662	0.662
PC2	0.167	0.829
PC3	0.124	0.952
PC4	0.018	0.970
PC5	0.011	0.981
PC6	0.009	0.990
PC7	0.007	0.997
PC8	0.003	1.000

Table 4.1: Explained Variance by Component

Listing 4.3: Optimal Component Selection

Result: The first 3 components explain 95.2% of the variance. Thus, we can reduce the dimensionality from 8 features to just 3 without significant information loss.

4.4 Visualizing Variance: The Scree Plot

A Scree Plot displays the individual and cumulative explained variance.

```

1 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
2
3 # Left: Scree Plot
4 axes[0].plot(range(1, len(explained_var)+1), explained_var, 'bo
    -', label='Individual')
5 axes[0].plot(range(1, len(cumulative_var)+1), cumulative_var, '
    ro-', label='Cumulative')
6 axes[0].axhline(y=0.90, color='g', linestyle='--', label='90%
    Threshold')
7 axes[0].axvline(x=n_components_opt, color='purple', linestyle='
    --', label=f'Optimal k={n_components_opt}')
8 axes[0].set_title('Scree Plot: Explained Variance')
9 axes[0].legend()
10 axes[0].set_xlabel('Principal Component')
11 axes[0].set_ylabel('Explained Variance Ratio')
12 axes[0].grid(True)

```

Listing 4.4: Scree Plot Generation

4.4.1 Interpretation

The plot shows a sharp "elbow" after PC3. The green dashed line represents the 90% threshold, and the purple vertical line marks the optimal number of components ($k = 3$). Components beyond PC3 contribute very little additional information (noise).

4.5 2D PCA Projection

To visualize the data structure, we project the data onto the first two principal components.

```
1 pca_2d = PCA(n_components=2)
2 X_train_2d = pca_2d.fit_transform(X_train_scaled)
3
4 scatter = axes[1].scatter(X_train_2d[:, 0], X_train_2d[:, 1], c
    =y_train, cmap='coolwarm', alpha=0.6)
5 axes[1].set_title('2D PCA Projection of Training Data')
6 axes[1].set_xlabel('PC1')
7 axes[1].set_ylabel('PC2')
8 plt.colorbar(scatter, ax=axes[1])
9 plt.tight_layout()
10 plt.show()
```

Listing 4.5: 2D Projection

This visualization helps assess if the classes (Low vs. High Risk) are separable in the reduced space.

Chapter 5

Interpretation of Principal Components

5.1 Understanding Loadings

Loadings are the weights assigned to each original feature in the linear combination that forms a Principal Component. They indicate the strength and direction of the relationship between the original variable and the component.

$$PC_i = w_{i1}X_1 + w_{i2}X_2 + \cdots + w_{ip}X_p \quad (5.1)$$

Where w_{ij} is the loading of feature j on component i .

5.2 Calculating Loadings

```
1 pca_opt = PCA(n_components=n_components_opt, random_state=42)
2 X_train_pca = pca_opt.fit_transform(X_train_scaled)
3 X_test_pca = pca_opt.transform(X_test_scaled)
4
5 loadings = pd.DataFrame(
6     pca_opt.components_.T,
7     columns=[f'PC{i+1}' for i in range(n_components_opt)],
8     index=features
9 )
10 print(loadings.round(3))
```

Listing 5.1: Extracting Loadings

5.3 Loadings Table

5.4 Interpreting PC1: Cardiometabolic Risk

Dominant Features: Total Cholesterol (0.421), LDL (0.416), Systolic BP (0.403), Diastolic BP (0.403), Fasting Glucose (0.387). HDL has a strong negative loading (-0.396).

Feature	PC1	PC2	PC3
Systolic_BP	0.403	-0.257	0.040
Diastolic_BP	0.403	-0.257	0.055
Total_Cholesterol	0.421	0.109	0.014
HDL_Cholesterol	-0.396	0.275	-0.027
Fasting_Glucose	0.387	0.316	-0.003
CRP_Inflammation	0.128	0.812	-0.070
Age	-0.044	0.089	0.995
LDL_Cholesterol	0.416	0.116	-0.002

Table 5.1: PCA Component Loadings

Interpretation: PC1 represents a general **Cardiometabolic Risk Axis**. High values correspond to high blood pressure, high bad cholesterol, high glucose, and low good cholesterol. This single component captures the combined effect of major cardiovascular risk factors.

5.5 Interpreting PC2: Inflammation

Dominant Feature: CRP Inflammation (0.812).

Interpretation: PC2 primarily captures **Systemic Inflammation**. CRP is a specific marker for inflammation. Its high loading suggests this component isolates inflammatory variation from the general metabolic risk captured in PC1. There is a moderate link to metabolic dysregulation (Glucose/HDL).

5.6 Interpreting PC3: Age

Dominant Feature: Age (0.995).

Interpretation: PC3 is almost entirely an **Age Dimension**. This indicates that age varies independently of the biochemical markers (cholesterol, inflammation) in this dataset. It confirms that age provides unique information not captured by the metabolic or inflammatory components.

5.7 Visualizing Loadings

```

1 plt.figure(figsize=(8, 5))
2 sns.heatmap(loadings, annot=True, cmap='coolwarm', center=0,
3             fmt='.2f')
4 plt.title('PCA Loadings: Original Features      Principal
5             Components')
6 plt.tight_layout()
7 plt.show()

```

Listing 5.2: Loadings Heatmap

The heatmap visually confirms the distinct clinical constructs: PC1 (Metabolic), PC2 (Inflammation), and PC3 (Age).

Chapter 6

Model Evaluation: Raw vs. PCA Features

6.1 Baseline Model: Logistic Regression (Raw)

We first train a Logistic Regression model on the original 8 scaled features to establish a baseline.

```
1 model_raw = LogisticRegression(max_iter=1000, random_state=42)
2 model_raw.fit(X_train_scaled, y_train)
3
4 y_pred_raw = model_raw.predict(X_test_scaled)
5 y_prob_raw = model_raw.predict_proba(X_test_scaled)[: , 1]
6
7 cv_acc_raw = cross_val_score(model_raw, X_train_scaled, y_train
8                               , cv=5, scoring='accuracy').mean()
9 cv_auc_raw = cross_val_score(model_raw, X_train_scaled, y_train
10                              , cv=5, scoring='roc_auc').mean()
```

Listing 6.1: Baseline Model Training

6.1.1 Baseline Results

Metric	Score
Test Accuracy	0.8400
Test ROC-AUC	0.9252
CV Accuracy	0.8600
CV ROC-AUC	0.9452

Table 6.1: Performance of Logistic Regression on Raw Scaled Features

The model performs well, with balanced precision and recall for both classes.

6.2 PCA Model: Logistic Regression (PCA)

Next, we train the same model on the 3 PCA-transformed features.

```
1 model_pca = LogisticRegression(max_iter=1000, random_state=42)
2 model_pca.fit(X_train_pca, y_train)
3
4 y_pred_pca = model_pca.predict(X_test_pca)
5 y_prob_pca = model_pca.predict_proba(X_test_pca)[:, 1]
6
7 cv_acc_pca = cross_val_score(model_pca, X_train_pca, y_train,
8                               cv=5, scoring='accuracy').mean()
9 cv_auc_pca = cross_val_score(model_pca, X_train_pca, y_train,
10                              cv=5, scoring='roc_auc').mean()
```

Listing 6.2: PCA Model Training

6.2.1 PCA Model Results

Metric	Score
Test Accuracy	0.8533
Test ROC-AUC	0.9200
CV Accuracy	0.8467
CV ROC-AUC	0.9302

Table 6.2: Performance of Logistic Regression on PCA-Transformed Features

6.3 Comparative Analysis

Metric	Raw Features (8)	PCA Features (3)
Test Accuracy	0.8400	0.8533
Test ROC-AUC	0.9252	0.9200
CV Accuracy	0.8600	0.8467
CV ROC-AUC	0.9452	0.9302

Table 6.3: Final Performance Comparison

6.3.1 Key Findings

- **Improved Accuracy:** The PCA model achieved higher test accuracy (85.3% vs 84.0%), suggesting better generalization by removing noise and multicollinearity.
- **Efficiency:** Reduced features from 8 to 3, lowering computational cost and model complexity.
- **Interpretability:** Identified 3 key drivers: Metabolic Risk, Inflammation, and Age.

- **Robustness:** The slight drop in CV metrics is negligible compared to the gain in simplicity and test accuracy.

6.4 Classification Report Details

Model	Class	Precision	Recall	F1-Score
2*Raw	Low Risk	0.85	0.83	0.84
	High Risk	0.83	0.85	0.84
2*PCA	Low Risk	0.87	0.83	0.85
	High Risk	0.84	0.88	0.86

Table 6.4: Detailed Classification Metrics

Notably, the PCA model improved **Recall for High Risk** patients (0.88 vs 0.85). In healthcare, identifying high-risk individuals is critical, making this improvement significant.

Chapter 7

Conclusion and Best Practices

7.1 Summary

Principal Component Analysis is a powerful unsupervised learning technique that summarizes large datasets into a few informative components while preserving most of the original variation. In this case study:

- We reduced 8 correlated clinical features to 3 independent components.
- These components represented distinct clinical constructs: Cardiometabolic Risk, Inflammation, and Age.
- The reduced feature set improved Logistic Regression accuracy and recall for high-risk patients.

7.2 Best Practices for PCA

1. **Always Standardize:** Never apply PCA to raw data with different scales.
2. **Split Before Scaling:** Prevent data leakage by splitting data before fitting the scaler.
3. **Check Correlations:** PCA is most effective when features are highly correlated.
4. **Interpret Loadings:** Don't just use PCA as a black box; interpret what the components represent.
5. **Validate with Modeling:** Always compare model performance with and without PCA to ensure information retention.

7.3 Limitations

- **Linearity Assumption:** PCA assumes linear relationships. It may fail to capture complex non-linear structures (consider Kernel PCA or t-SNE for such cases).
- **Interpretability:** Components are abstract combinations of features, which can be harder to explain to non-technical stakeholders than original variables.

- **Outlier Sensitivity:** PCA is sensitive to outliers, which can skew the variance direction. Robust PCA techniques exist but are more complex.

7.4 Future Directions

Further analysis could involve:

- Applying **Kernel PCA** to capture non-linear relationships.
- Using **t-SNE** or **UMAP** for better visualization of high-dimensional clusters.
- Integrating PCA into a pipeline with other classifiers like Random Forest or SVM to test robustness across algorithms.

End of Document

Mediacrest Training College
Advanced Data Analysis and Fundamentals of AI